Google AI for Developers

Pluralsight Course — Full Revision Notes

This document covers all 5 modules of the course — Gemini API, Model Selection, Prompt Engineering, Gemini Integration, and Security & Cost Optimization — along with the demo code walkthroughs. Use it to revise everything end to end.

Module 1 — Implementing Google Gemini API Workflows

What is Google Gemini?

Gemini is Google's cutting-edge multimodal generative AI. It can handle inputs and outputs across multiple modes — text, images, audio, and video — in a single model. It runs on Google Cloud infrastructure (TPUs) and is accessed through the Google API gateway.

> **Key point:** With the free tier, Google uses your data to improve its products. For production/enterprise work, use Vertex AI to keep data private.

Gemini Model Versions (as of 2025)

Model	Release	Context	Speed	Thinking	Best For
Gemini 2.5 Pro	March 2025	1M tokens	Standard	Yes	Complex tasks, coding, deep reasoning
Flash 2.5	April 2025	1M tokens	Faster	Yes	Reasoning tasks balancing speed and cost
Flash-Lite 2.0	Feb 2025	—	Fastest	No	Quick responses, simple queries, embedded apps

Thinking Capability

The 'thinking' feature lets the model allocate more computation to harder problems. Instead of always responding at the same speed, the model dynamically adjusts its reasoning time based on the complexity of the request — balancing speed versus depth of response.

Setting Up the API

- Go to https://ai.google.dev and get your API key
- Install the Gemini SDK: pip install google-genai
- Or use the HTTP API directly without installing anything

Optimizing API Requests

- Batch requests — send multiple prompts together instead of one at a time
- Async calls — do not block your app waiting for a response; run calls concurrently
- Streaming responses — receive tokens as they generate instead of waiting for the full output
- Optimize multimedia inputs — compress and resize images/audio before sending
- Efficient data handling — only send what the model actually needs

Multimodal Data Handling

Gemini can process text, images, audio, and video together in a single request. It converts all inputs into a unified embedding before processing.

Method	When to use	Limit
Inline data	Pass data directly in the request body	Files smaller than 20 MB
Files API	Upload the file first, then reference it by URI	When total request size exceeds 20 MB
Mixed	Combine inline and Files API references in one prompt	No restriction on mixing

---

Module 2 — Google Gemini Model Selection

How Gemini Works Internally

Gemini is based on the Transformer architecture — a large neural network with many layers. It is slow and costly at scale, which is why choosing the right model variant for each task matters. The data flow for every generation request is:

- Send input data to Gemini
- Input is converted into final input representation based on the prompt structure
- Model generates output based on the base model weights and the parameters you set

Choosing a Model — Two Dimensions

Dimension	Options	Trade-off
Size	Small (Flash-Lite) → Medium (Flash) → Large (Pro)	Speed vs Quality — bigger = smarter but slower
Stability	Preview (latest features, may change) vs Stable (production-safe)	Quality vs Predictability — preview = newer, stable = reliable

Model Parameters

Temperature

Controls the randomness of the model's output. It works by scaling the probability distribution over possible next tokens.

- **High temperature:** .g. 1.0+): probabilities become more equal — model picks less likely words more often. Output is more creative and varied but less accurate.
- **Low temperature:** .g. 0.1): probabilities spread further apart — model strongly favours the top choice. Output is more focused, deterministic, and repeatable.
- **Temperature = 0:** fully deterministic. Same input always gives the same output.

Top-P (Nucleus Sampling)

Controls the diversity of the output by limiting which tokens the model can choose from. Instead of considering all tokens, it only considers the smallest group of tokens whose cumulative probability adds up to P.

- Top-P = 0.9: only tokens in the top 90% of probability mass are considered
- Lower Top-P = less diverse, more focused output
- Higher Top-P = more diverse, wider vocabulary

> **Practical tip:** For code generation or factual Q&A, use low temperature (0–0.2) and lower Top-P. For creative writing, use higher temperature (0.7–1.0) and higher Top-P.

Prompt Structures

Structure	What it is	Best For
Freeform	Just type a question or instruction — no template	Zero-shot tasks, simple queries
Structured	Provide labeled sections: context, examples, instruction, output format	Tasks needing consistent output format
Chat	Multi-turn conversation with user/assistant roles	Dialogue, follow-up questions, agents

---

Module 3 — Prompt Engineering for Developers with Google Gemini

Why Prompt Engineering Matters for Developers

Gemini 2.5 Pro is especially powerful for coding tasks — finding bugs, writing unit tests, explaining code, and 'vibe coding' (generating code from natural language descriptions). How you write the prompt directly determines the quality of what you get back.

Technique 1 — Few-Shot Prompting

You include a few examples of input-output pairs inside the prompt before asking your actual question. The examples teach the model the pattern and format you want it to follow.

Structure of a few-shot prompt:

- Opening instruction — what you want the model to do
- Example 1 — a sample input with its ideal output
- Example 2 — another sample to reinforce the pattern
- Actual question — the real input you want the model to answer

From the demo (gemini.py):

```
from google import genai
client = genai.Client(api_key="YOUR_API_KEY")
response = client.models.generate_content(
    model="gemini-2.5-pro-exp-03-25",
    contents="""
Explain how AI works in a few words using bullet points.

Example 1:
Question: How does AI work?
Answer: Learns from data / Recognizes patterns / Makes predictions

Example 2:
Question: Can you briefly explain AI?
Answer: Uses algorithms / Identifies trends / Improves through training

Now, complete the following:
Question: Explain AI in a few words.
""")
```

> **Why it works:** The model infers the expected output format from your examples without you needing to describe the format explicitly. The examples do the teaching.

Technique 2 — Chain-of-Thought Prompting

You include an example where the model is shown not just the question and answer, but also the step-by-step reasoning that leads to the answer. This guides the model to reason through your actual problem the same way.

Structure:

- Problem statement — describe the issue or question
- Example reasoning — show the thought process for a similar problem
- Actual question — the problem you want solved

Chain-of-thought benefits:

- Model gives a step-by-step breakdown — easier to verify and debug
- Improves accuracy — less likely to jump to wrong conclusions on complex problems
- Especially useful for debugging code, multi-step calculations, and logic problems

Technique 3 — Context Priming

You give the model background information about the situation before asking the question. This primes the model so it answers from the right perspective — like briefing a colleague before asking for their opinion.

- Example: 'You are a senior Python developer reviewing a pull request for a REST API. The codebase uses FastAPI and PostgreSQL. Here is the code: [code]. What issues do you see?'
- The context (role, tech stack, task) shapes every part of the response

Using Gemini for Code Tasks

Task	How Gemini helps
Find code issues	Paste the code and ask it to identify bugs or anti-patterns
Add unit tests	Give it a function and ask it to write test cases covering edge cases
Explain code	Ask it to explain what a block of code does in plain language
Vibe coding	Describe what you want in words and let it generate the implementation
Improve code	Ask it to refactor for readability, performance, or best practices

---

Module 4 — Google Gemini Integration

Gemini 2.0 Innovations

- Native tool use — model can call functions and external services natively
- Multimodal output — model can generate text, code, and other modalities in one response
- Real-time streaming — output streams token by token for responsive user experiences

Google AI Studio vs Vertex AI Studio

Aspect	Google AI Studio	Vertex AI Studio
Target audience	Developers, researchers, beginners	Enterprise developers and engineers
Focus	Prototyping and experimentation	Prototyping within an enterprise context
MLOps / LLMOps	Limited	Full Vertex AI platform capabilities
Data privacy	Data may be used by Google (free tier)	Enterprise-grade, your data stays private

Aspect	Google AI Studio	Vertex AI Studio
Access method	API key	Service account / Application Default Credentials
Best for	Learning, quick prototypes	Production enterprise applications

Google Gen AI SDK — Two Ways to Connect

The same SDK supports both Google AI Studio (API key) and Vertex AI (enterprise credentials). You switch between them by setting vertexai=True/False.

Option 1 — Google AI Studio (API key)

```
from google import genai
client = genai.Client(vertexai=False, api_key="YOUR_API_KEY")
response = client.models.generate_content(
    model="gemini-2.0-flash-001",
    contents="Explain bubble sort to me.",
)
print(response.text)
```

Option 2 — Vertex AI (enterprise, uses Application Default Credentials)

```
from google import genai
client = genai.Client(vertexai=True)
response = client.models.generate_content(
    model="gemini-2.0-flash-001",
    contents="Explain bubble sort to me.",
)
print(response.text)
```

To authenticate for Vertex AI, run this once in your terminal:

```
gcloud auth application-default login
```

Environment variables needed for Vertex AI (from .env.example in the demo):

```
GOOGLE_CLOUD_PROJECT=YOUR_GOOGLE_CLOUD_PROJECT
GOOGLE_CLOUD_LOCATION=GOOGLE_CLOUD_REGION
GOOGLE_GENAI_USE_VERTEXAI=True
```

Demo Walkthrough — Gemini Agent with Tools

The demo.py file shows how to build a two-step agent using native tools — Google Search to get live data, and Code Execution to perform calculations on it.

Step 1 — Get exchange rate using Google Search tool

```
from google.genai.types import Tool, GenerateContentConfig, Content, Part, GoogleSearch

def get_exchange_rate(client, currency1, currency2):
    contents = [Content(role="user", parts=[
        Part.from_text(f"What is the exchange rate between {currency1} and {currency2}?")
    ])]
    tools = [Tool(google_search=GoogleSearch())]
    config = GenerateContentConfig(temperature=0.2, top_p=0.8,
                                   max_output_tokens=1024, tools=tools)
    response = client.models.generate_content(
        model=MODEL_ID, contents=contents, config=config)
    return response.text
```

Step 2 — Calculate conversion using Code Execution tool

```
from google.genai.types import Tool, ToolCodeExecution

def calculate_for(amount, rate):
    tool = Tool(code_execution=ToolCodeExecution())
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=f"Calculate {amount} times the provided rate: {rate}.",
        config=GenerateContentConfig(tools=[tool], temperature=0),
    )
    return response.executable_code, response.code_execution_result
```

Running the agent

```
client = genai.Client(vertexai=True)
rate = get_exchange_rate(client, "USD", "EUR")
code, result = calculate_for(512.0, rate)
print("Result:", result)
```

> **What this shows:** The model uses Google Search to get real-time data (exchange rate), then generates and executes Python code to do the calculation. Neither tool was hardcoded — the model decides when and how to use each one.

Frameworks and Tools That Work with Gemini

- LangChain / LangGraph — chain-based and graph-based agent orchestration
- LlamaIndex — document indexing and RAG pipelines
- CrewAI — role-based multi-agent teams
- LiteLLM — unified interface to switch between multiple LLM providers
- OpenRouter — route requests across multiple AI models and providers

---

Module 5 — Google Gemini Security and Cost Optimization

Part A — Data Privacy in Google Gemini

How Google Uses Your Data

Scenario	What happens to your data
Free tier (AI Studio)	Data may be used by Google to improve its products
Gemini as mobile assistant / with extensions	Interactions are processed to understand your needs
Gemini for content creation	Content you generate may be reviewed
Vertex AI / Enterprise	Enterprise-grade security — Google does not use your data for training

Data Retention — When Gemini App Activity is ON

- Conversations are saved to your Google account
- Google may use saved activity to improve products
- You can delete your activity at any time from your Google account settings

Data Retention — When Gemini App Activity is OFF

- Future chats will not be reviewed by Google
- Chats are saved in your account for up to 72 hours only, then deleted

- Google may still save location and other data separately
- Past chats remain until you delete them manually

Key Regulations to Know

Regulation	Region	Rights it gives users
GDPR (General Data Protection Regulation)	European Union	Right to access data, right to correct data, right to erase data. Brings transparency to data collection.
CCPA (California Consumer Privacy Act)	California, USA	Right to know what data is collected, right to opt-out of data sale, right to request deletion.

> **Key takeaway:** Both GDPR and CCPA give you control and visibility over your personal data. Knowing these regulations helps you understand what Google is and is not allowed to do with your interactions.

Part B — Key Drivers for Gemini's Performance

Model Size vs Model Complexity

These are two different things that are often confused:

- **Model size —** the number of parameters in the model. More parameters = more knowledge capacity but slower and more expensive.
- **Model complexity —** the depth and architecture of the model (how many layers, attention heads, etc.). A model can be large but simple, or small but complex.

Factor	Effect on Inference Time	Effect on Resource Usage
Larger model size	Slower — more operations per forward pass	Higher — needs more GPU memory and compute
Higher model complexity	Slower — more layers = more sequential computation	Higher — memory grows with depth
Smaller / simpler model	Faster — fewer operations	Lower — fits on cheaper hardware

> **Practical point:** Bigger and more complex models are not always better for production. For real-time applications, a smaller, faster model that is 90% as accurate is often the right choice over a slow but perfect one.

Input Data Quality — Its Impact on Gemini

Think of Gemini as a student — its knowledge and abilities are shaped by the information it is given. This applies both during training and during inference.

During Training

- Data accuracy and precision — wrong training data leads to wrong outputs
- Data relevance — off-topic data wastes model capacity
- Data completeness — gaps in training data create gaps in model knowledge
- Unbiased data — biased training data leads to biased model behavior

During Inference (when you send a prompt)

- Input quality — a vague or poorly written prompt gives vague output
- Context and detail — more context means the model can give a more accurate, relevant answer
- Task understanding — the clearer you define the task, the better the model understands what you want

Techniques for Optimizing API Requests

Think of the Gemini API as a communication channel — clarity and efficiency matter on both ends.

Optimize your requests

- Clear and concise prompts — avoid ambiguity; be specific about what you want
- Batch requests — send multiple prompts in one API call when possible to reduce overhead
- Leverage parameters — use temperature, Top-P, and max_output_tokens to control output precisely

Optimize the response

- Specify response fields — if you only need specific parts of the output, say so in the prompt
- Filter and transform data — process and clean the response before passing it downstream
- Implement caching — cache responses to identical or near-identical queries to avoid repeat API calls
- Compression — compress large responses when storing or transmitting them

Bonus tips

- Stay updated — check the latest API documentation; Gemini updates frequently
- Implement robust error handling — always handle 429 (rate limit), 500, and timeout errors gracefully

---

Revision Questions

Module 1 — Gemini API

Q1. What are the three Gemini model variants released in early 2025, and what is each one best used for?

Q2. What does the 'thinking' capability in Gemini 2.5 Pro actually do? How does it decide how much to think?

Q3. What are the two ways to send file data to Gemini, and when would you use each?

Q4. Name three techniques for optimizing API requests when working with Gemini at scale.

Q5. What is the data policy difference between using Gemini's free tier vs Vertex AI?

Module 2 — Model Selection

Q6. Explain the difference between temperature and Top-P. What happens to output when you increase temperature?

Q7. You are building a chatbot that needs to return consistent, formatted JSON every time. What temperature and Top-P settings would you use and why?

Q8. What are the three prompt structures in Gemini? Give a real-world example of when you would use each one.

Q9. If a model produces different output every time you run the same prompt, which parameter is most likely causing that? How would you fix it?

Module 3 — Prompt Engineering

Q10. What is few-shot prompting? Write a brief example structure for a few-shot prompt that teaches Gemini to convert a sentence to formal English.

Q11. What is chain-of-thought prompting and why does it improve accuracy on complex problems?

Q12. What is context priming? How would you prime Gemini to help you debug a FastAPI Python application?

Q13. Name four coding tasks that Gemini 2.5 Pro is especially useful for.

Q14. What is the difference between using few-shot prompting and just giving Gemini a detailed instruction without examples?

Module 4 — Gemini Integration

Q15. What is the difference between Google AI Studio and Vertex AI Studio? When should you choose each?

Q16. Show how you would initialize a genai.Client for Vertex AI vs for Google AI Studio. What changes between the two?

Q17. In the demo.py code, the agent uses two tools. What are they and what does each one do?

Q18. What command do you run to authenticate for Vertex AI with Application Default Credentials?

Q19. What three environment variables does the demo need when using Vertex AI? What does each one do?

Q20. Name three frameworks that work with Gemini and briefly describe what each one is used for.

Q21. What are the three Gemini 2.0 innovations mentioned in the course?

Module 5 — Security and Cost Optimization

Q22. What happens to your conversation data when Gemini App Activity is turned ON vs turned OFF?

Q23. What rights does GDPR give users regarding their data? Name three.

Q24. What is the difference between model size and model complexity? How does each affect inference time?

Q25. A company is building a real-time customer support chatbot and is deciding between Gemini 2.5 Pro and Flash-Lite 2.0. Which would you recommend and why?

Q26. What four properties of training data affect a model's quality? Why does unbiased data matter?

Q27. Name four techniques for optimizing API responses (not requests).

Q28. You notice your app is repeatedly calling the Gemini API with the same question from different users. What is the most cost-effective fix?

Q29. What is the difference between GDPR and CCPA in terms of which regions they apply to and what rights they provide?

Q30. If your Gemini API calls are returning slow responses and high costs, name three specific things you can do to improve both.